Student Performance Analytics System

1. Introduction

The Student Performance Analytics System is a Python-based project designed to analyze students' academic performance using their subject marks and attendance records.

Analyzing student records manually can take time, especially when the dataset contains many students. This project uses Python, NumPy, and Pandas to organize student data and generate useful academic performance summaries.

2. Objectives

- Create and manage a student dataset in CSV format.
- Load and organize data using Pandas.
- Perform numerical calculations using NumPy.
- Calculate total and average marks for each student.
- Assign grades and determine pass/fail results using defined conditions.
- Analyze subject-wise performance.
- Identify the highest-performing, lowest-performing, and top-performing students.
- Practice Python variables, data types, loops, and functions.

3. Technologies Used

- **Python:** Programming and data processing.
- **NumPy:** Numerical calculations.
- **Pandas:** Data organization and analysis.
- **Google Colab:** Environment for writing and running Python code.
- **CSV:** Format used to store student records.

4. Expected Outcome

The project will generate an organized student dataset and meaningful academic performance summaries.

Note: The dataset will contain synthetic student records created for educational purposes.

5. Dataset Creation

A dataset is a collection of related information organized in rows and columns. In this project, each row represents one student, and each column represents a student attribute, such as Student ID, Name, Department, subject marks, or Attendance.

The dataset will contain 500 synthetic student records. We will use Python to generate the records, NumPy to help create numerical values, and Pandas to organize and save the data in CSV format.

The generated data will be used to calculate student totals, averages, grades, pass/fail results, and subject-wise performance.

5.1 Importing Libraries

Python libraries are collections of prewritten code that help us perform specific tasks.

In this project, we will use:

- **Pandas:** To create tables, organize student records, and save data to a CSV file.
- **NumPy:** To generate and process numerical data, including student marks.
- **Pathlib:** To manage the project folder and file paths.

```
import pandas as pd
import numpy as np
from pathlib import Path

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

5.2 Creating the Project Folder

A project folder keeps all files related to a project organized in one place.

We will create a folder named `Student_Performance_Analytics`. Our dataset file, `students.csv`, will be saved inside this folder.

We will use Python's `pathlib` library to create and manage the folder path. This makes it easier to save and locate project files.

```
from pathlib import Path

# Create the main project folder
project = Path("/content/Student_Performance_Analytics")
project.mkdir(parents=True, exist_ok=True)

# Create the required project files
files = [
    "main.py",
    "students.csv",
    "functions.py",
    "README.md"
]

for filename in files:
    (project / filename).touch(exist_ok=True)

print("Project folder and files created successfully!")

# Display the file structure
for file in sorted(project.iterdir()):
    print(file.name)
```

```
Project folder and files created successfully!
README.md
functions.py
grade_distribution_report.csv
main.py
student_performance_report.csv
```

```
students.csv
subject_performance_report.csv
```

5.3 Defining the Dataset Size

We will create records for 500 students. Defining the dataset size in a variable makes it easy to change the number of records later without modifying the rest of the program.

```
# Define the number of student records
number_of_students = 500

print("Number of student records:", number_of_students)
```

```
Number of student records: 500
```

5.4 Creating Student Name Lists

Each student record should contain a name. To generate synthetic student records, we will create two Python lists: one containing first names and another containing last names.

Python lists allow us to store multiple values under a single variable name. Later, we can combine a first name and a last name to generate a student's full name.

These names are synthetic examples created for this educational project and do not represent verified real student records.

```
# List of student first names
first_names = [
    "Aarav", "Aditya", "Akash", "Amit", "Ananya",
    "Aniket", "Arjun", "Asha", "Bhavana", "Chetan",
    "Deepak", "Diya", "Gaurav", "Isha", "Karan",
    "Kavya", "Krishna", "Meera", "Neha", "Nikhil",
    "Pooja", "Pranav", "Priya", "Rahul", "Riya",
    "Rohan", "Sakshi", "Sameer", "Sneha", "Tanvi",
    "Varun", "Vedant", "Vikram", "Yash", "Zoya"
]

# List of student last names
last_names = [
    "Patil", "Shinde", "Deshmukh", "Jadhav", "Kulkarni",
    "Joshi", "Pawar", "More", "Kadam", "Chavan",
    "Gaikwad", "Mane", "Pillai", "Reddy", "Sharma",
    "Verma", "Gupta", "Khan", "Singh", "Mishra",
    "Nair", "Iyer", "Mehta", "Bhosale", "Thakur"
]

print("Number of first names:", len(first_names))
print("Number of last names:", len(last_names))
print("Example full name:", first_names[0], last_names[0])
```

```
Number of first names: 35
Number of last names: 25
Example full name: Aarav Patil
```

5.5 Creating the Department List

Students may belong to different academic departments. To represent this information in our dataset, we will create a Python list containing five department names.

Each student record will be assigned a department from this list. This will allow us to compare academic performance across departments during the analysis stage.

```
# List of academic departments
departments = [
    "ENTC",
    "Computer Engineering",
    "Information Technology",
    "Electrical Engineering",
    "Mechanical Engineering"
]

print("Available departments:")

for department in departments:
    print(department)

print("\nTotal departments:", len(departments))
```

```
Available departments:
ENTC
Computer Engineering
Information Technology
Electrical Engineering
Mechanical Engineering

Total departments: 5
```

5.6 Creating the Subject List

Each student will have marks for five subjects: Python, Mathematics, Artificial Intelligence (AI), Statistics, and Communication.

We will store the subject names in a Python list. This makes it easier to manage the subjects and perform calculations such as total marks, average marks, and subject-wise performance.

Each subject will have a maximum score of 100 marks.

```
# List of subjects
subjects = [
    "Python",
    "Mathematics",
    "AI",
    "Statistics",
    "Communication"
]

print("Subjects included in the dataset:")

for subject in subjects:
    print(subject)

print("\nTotal number of subjects:", len(subjects))
print("Maximum marks per subject: 100")
```

```
Subjects included in the dataset:
Python
Mathematics
AI
Statistics
Communication

Total number of subjects: 5
Maximum marks per subject: 100
```

5.7 Generating Student Marks Using NumPy

NumPy is a Python library used for numerical calculations. In this project, we will use NumPy to generate synthetic marks for students.

Students should not all receive the same marks. Therefore, we will generate different marks to represent different academic performance levels.

For this educational dataset, we will use three performance groups:

- **High Performance:** Students generally receive higher marks.
- **Average Performance:** Students generally receive moderate marks.
- **Needs Improvement:** Students generally receive lower marks.

These groups are used only to generate varied marks. The final grades and pass/fail results will be calculated later from the actual subject marks.

```
import numpy as np

# Create a NumPy random number generator
rng = np.random.default_rng(42)

# Generate five sample marks using NumPy
sample_marks = rng.integers(40, 101, size=5)

print("Sample student marks:", sample_marks)
```

```
Sample student marks: [45 87 79 66 66]
```

5.8 Defining Student Performance Groups

Students can have different levels of academic performance. To make our synthetic dataset more useful for analysis, we will define three performance groups: High, Average, and Needs Improvement.

We will use NumPy to randomly assign each student to one of these groups. Each group will have a different typical mark level.

These groups are used only to generate varied sample data. They will not determine the final grade or pass/fail result. Those results will be calculated later using the student's actual marks and the project's defined rules.

```
# Set the number of students in our dataset
number_of_students = 500

# Randomly assign a performance group to each student
```

```
performance_groups = rng.choice(
    ["High", "Average", "Needs Improvement"],
    size=number_of_students,
    p=[0.25, 0.50, 0.25]
)

print("Total performance-group assignments:", len(performance_groups))
print("\nFirst 10 assignments:")
print(performance_groups[:10])

print("\nNumber of students in each group:")
print(pd.Series(performance_groups).value_counts())
```

```
Total performance-group assignments: 500

First 10 assignments:
['Average' 'High' 'Needs Improvement' 'Needs Improvement'
 'Needs Improvement' 'High' 'Average' 'Average' 'Needs Improvement'
 'Average']

Number of students in each group:
Average              244
High                 132
Needs Improvement    124
Name: count, dtype: int64
```

5.9 Defining Typical Marks for Each Performance Group

Different performance groups will have different typical mark levels.

We will use a Python dictionary to store the typical mark level for each group. A dictionary stores information as key-value pairs.

For this dataset, we will use the following approximate mark levels:

- High: 84 marks
- Average: 66 marks
- Needs Improvement: 42 marks

NumPy will use these values when generating marks for individual subjects. The actual marks will vary between students and subjects.

These values are assumptions for synthetic data generation, not results from real student records.

```
# Define the typical mark level for each performance group
group_means = {
    "High": 84,
    "Average": 66,
    "Needs Improvement": 42
}

print("Typical marks for each performance group:")

for group, mean_mark in group_means.items():
    print(group, ":", mean_mark)
```

```
Typical marks for each performance group:
High : 84
Average : 66
Needs Improvement : 42
```

5.10 Generating Subject Marks for One Student

We will use NumPy's normal distribution to generate marks around the typical mark level of a performance group.

The `loc` parameter specifies the center of the distribution, while `scale` controls how much the generated values vary. We will round the marks to whole numbers and keep them within the range of 0 to 100.

```
# Select the first student's performance group
group = performance_groups[0]

# Find the typical mark level for that group
ability = group_means[group]

# Generate marks for five subjects
sample_student_marks = rng.normal(
    loc=ability,
    scale=11,
    size=len(subjects)
)

# Round marks and keep them between 0 and 100
sample_student_marks = np.clip(
    np.round(sample_student_marks),
    0,
    100
).astype(int)

print("Performance group:", group)
print("Typical mark level:", ability)
print("Subjects:", subjects)
print("Generated marks:", sample_student_marks)
```

```
Performance group: Average
Typical mark level: 66
Subjects: ['Python', 'Mathematics', 'AI', 'Statistics', 'Communication']
Generated marks: [65 64 57 71 57]
```

5.11 Structure of a Student Record

Each student record will contain a unique Student ID, full name, department, marks in five subjects, and attendance percentage.

We will store each student's information as a Python dictionary. A dictionary allows us to associate each field name with its corresponding value.

For example, the field `Python` stores the student's Python marks, while `Attendance` stores the student's attendance percentage.

We will later combine these individual records into a Pandas DataFrame and save them in CSV format.

```
# Create sample data if the variables are not already defined
first_names = ["Aarav", "Priya", "Rahul", "Sneha", "Neha"]
last_names = ["Patil", "Sharma", "Deshmukh", "Joshi", "Kulkarni"]
```

```
departments = [
    "ENTC",
    "Computer Engineering",
    "Information Technology"
]

# Create a sample list of student names
student_names = [
    first + " " + last
    for first, last in zip(first_names, last_names)
]

# Create one sample student record
sample_record = {
    "Student_ID": "STU0001",
    "Name": student_names[0],
    "Department": departments[0],
    "Python": 78,
    "Mathematics": 82,
    "AI": 75,
    "Statistics": 80,
    "Communication": 85,
    "Attendance": 90
}

print("Sample student record:")
print(sample_record)
```

```
Sample student record:
{'Student_ID': 'STU0001', 'Name': 'Aarav Patil', 'Department': 'ENTC', 'Pythc
```

5.12 Generating the Complete Student Dataset

We will use a `for` loop to generate 500 student records. For each student, Python will assign a unique ID, select a name and department, generate marks for five subjects, and assign an attendance percentage.

Each completed record will be added to a list. After the loop finishes, the list will contain all 500 student records.

```
print("Does student_names exist?", "student_names" in globals())
```

```
Does student_names exist? True
```

```
# Re-define the complete list of names to ensure we have enough unique combi
first_names = [
    "Aarav", "Aditya", "Akash", "Amit", "Ananya",
    "Aniket", "Arjun", "Asha", "Bhavana", "Chetan",
    "Deepak", "Diya", "Gaurav", "Isha", "Karan",
    "Kavya", "Krishna", "Meera", "Neha", "Nikhil",
    "Pooja", "Pranav", "Priya", "Rahul", "Riya",
    "Rohan", "Sakshi", "Sameer", "Sneha", "Tanvi",
    "Varun", "Vedant", "Vikram", "Yash", "Zoya"
]

last_names = [
    "Patil", "Shinde", "Deshmukh", "Jadhav", "Kulkarni",
    "Joshi", "Pawar", "More", "Kadam", "Chavan",
    "Gaikwad", "Mane", "Pillai", "Reddy", "Sharma",
```

```
    "Verma", "Gupta", "Khan", "Singh", "Mishra",
    "Nair", "Iyer", "Mehta", "Bhosale", "Thakur"
]

# Create all possible full-name combinations (35 * 25 = 875 combinations)
all_name_combinations = [
    first + " " + last
    for first in first_names
    for last in last_names
]

# Check how many unique names are available
print("Available unique names:", len(all_name_combinations))

# Select 500 unique names safely
number_of_students = 500

student_names = rng.choice(
    all_name_combinations,
    size=number_of_students,
    replace=False
).tolist()

print("Total student names:", len(student_names))
print("First five names:", student_names[:5])
```

```
Available unique names: 875
Total student names: 500
First five names: ['Diya Mishra', 'Kavya Pawar', 'Yash Khan', 'Amit Pawar', '
```

```
# Create one sample student record

i = 0

sample_record = {
    "Student_ID": f"STU{i + 1:04d}",
    "Name": student_names[i]
}

print(sample_record)
```

```
{'Student_ID': 'STU0001', 'Name': 'Diya Mishra'}
```

```
sample_record = {
    "Student_ID": f"STU{i + 1:04d}",
    "Name": student_names[i]
}
```

5.12 Generating Student Records Using a Loop

A loop allows us to repeat the same operation multiple times. In this project, we will use a `for` loop to generate records for 500 students.

For every student, the program will assign a unique Student ID, a name, a department, marks in five subjects, and an attendance percentage.

Each student's information will be stored in a dictionary. The dictionaries will be collected in a Python list called `records`, which will later be converted into a Pandas DataFrame.

```
# Create an empty list to store student records
records = []

# Generate records for all students
for i in range(number_of_students):

    # Get this student's performance group
    group = performance_groups[i]

    # Get the typical marks for that group
    ability = group_means[group]

    # Generate marks for the five subjects
    marks = rng.normal(
        loc=ability,
        scale=11,
        size=len(subjects)
    )

    # Round marks and restrict them to 0–100
    marks = np.clip(
        np.round(marks),
        0,
        100
    ).astype(int)

    # Create the student's complete record
    record = {
        "Student_ID": f"STU{i + 1:04d}",
        "Name": student_names[i],
        "Department": rng.choice(departments),
        "Python": marks[0],
        "Mathematics": marks[1],
        "AI": marks[2],
        "Statistics": marks[3],
        "Communication": marks[4],
        "Attendance": int(rng.integers(50, 101))
    }

    # Add the record to the list
    records.append(record)

print("Total student records generated:", len(records))
print("\nFirst student record:")
print(records[0])
```

```
Total student records generated: 500

First student record:
{'Student_ID': 'STU0001', 'Name': 'Diya Mishra', 'Department': np.str_('ENTC'
```

5.13 Creating a Pandas DataFrame

A Pandas DataFrame is a two-dimensional data structure arranged in rows and columns, similar to a spreadsheet.

In the previous step, we stored each student's information in a dictionary and collected the dictionaries in a list called `records`.

Now, we will convert that list into a DataFrame. This will make it easier to view, organize, and analyze the student data.

```
# Convert the list of student records into a DataFrame
df = pd.DataFrame(records)

# Display the first five student records
display(df.head())
```

	Student_ID	Name	Department	Python	Mathematics	AI	Statistics	Commur
0	STU0001	Diya Mishra	ENTC	55	55	78	61	
1	STU0002	Kavya Pawar	Computer Engineering	89	67	87	92	
2	STU0003	Yash Khan	Computer Engineering	27	46	39	37	

```
# Display the number of rows and columns
print("Dataset dimensions:", df.shape)

print("Number of students:", df.shape[0])
print("Number of columns:", df.shape[1])
```

```
Dataset dimensions: (500, 9)
Number of students: 500
Number of columns: 9
```

5.15 Saving the Dataset as a CSV File

CSV stands for Comma-Separated Values. It is a file format used to store tabular data in rows and columns.

We will use Pandas to save our DataFrame as `students.csv` inside the project folder. This file can be opened in spreadsheet applications and loaded into Python for future analysis.

We will use `index=False` to prevent Pandas from saving its row index as an additional column.

```
# Define the CSV file path
csv_path = project / "students.csv"

# Save the DataFrame as a CSV file
df.to_csv(csv_path, index=False)

print("Dataset saved successfully!")
print("File location:", csv_path)
```

```
Dataset saved successfully!
File location: /content/Student_Performance_Analytics/students.csv
```

5.16 Verifying the CSV File

After saving the dataset, we will check whether the file exists and reload it using Pandas. This helps confirm that the data has been saved and can be used for analysis.

```
# Check whether the CSV file exists
print("File exists:", csv_path.exists())

# Load the saved CSV file
df_check = pd.read_csv(csv_path)

# Check the loaded data
print("Loaded dataset dimensions:", df_check.shape)

# Display the first five rows
display(df_check.head())
```

```
File exists: True
Loaded dataset dimensions: (500, 9)
```

	Student_ID	Name	Department	Python	Mathematics	AI	Statistics	Commur
0	STU0001	Diya Mishra	ENTC	55	55	78	61	
1	STU0002	Kavya Pawar	Computer Engineering	89	67	87	92	
2	STU0003	Yash Khan	Computer Engineering	27	46	39	37	

6. Data Inspection and Quality Checking

Data inspection is the process of examining a dataset to understand its structure and check whether the data is suitable for analysis.

In this project, we will use Pandas to inspect the student records, identify the columns, check data types, and look for missing or duplicate values.

Checking data quality before performing calculations helps us avoid incorrect results and makes our analysis more reliable.

6.1 Loading the Dataset

We will load the saved `students.csv` file using Pandas. This ensures that our analysis uses the data stored in the CSV file rather than relying only on the original Python list.

```
# Load the saved student dataset
df = pd.read_csv(project / "students.csv")

# Display the first five records
display(df.head())
```

	Student_ID	Name	Department	Python	Mathematics	AI	Statistics	Commur
0	STU0001	Diya Mishra	ENTC	55	55	78	61	
1	STU0002	Kavya Pawar	Computer Engineering	89	67	87	92	
2	STU0003	Yash Khan	Computer Engineering	27	46	39	37	

6.2 Examining Dataset Structure

We will examine the dataset's dimensions, column names, and data types. This helps us confirm how many records and attributes are present and whether each column contains the expected kind of information.

```
# Display the number of rows and columns
print("Dataset dimensions:", df.shape)

# Display all column names
print("\nColumn names:")
print(df.columns.tolist())

# Display column data types and non-null counts
print("\nDataset information:")
df.info()
```

```
Dataset dimensions: (500, 9)

Column names:
['Student_ID', 'Name', 'Department', 'Python', 'Mathematics', 'AI', 'Statisti

Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Student_ID     500 non-null    object
 1   Name           500 non-null    object
 2   Department     500 non-null    object
 3   Python         500 non-null    int64
 4   Mathematics    500 non-null    int64
 5   AI             500 non-null    int64
 6   Statistics     500 non-null    int64
 7   Communication  500 non-null    int64
 8   Attendance     500 non-null    int64
dtypes: int64(6), object(3)
memory usage: 35.3+ KB
```

6.3 Checking Missing Values

Missing values occur when a field has no recorded value. For example, a student might have a blank Mathematics mark or missing attendance information.

We will use Pandas to count missing values in each column. This helps us identify whether any records need correction before analysis.

```
# Count missing values in each column
print("Missing values per column:")
print(df.isnull().sum())

# Count missing values in the entire dataset
print("\nTotal missing values:", df.isnull().sum().sum())
```

```
Missing values per column:
Student_ID       0
Name             0
Department       0
```

```
Python           0
Mathematics      0
AI               0
Statistics       0
Communication    0
Attendance       0
dtype: int64

Total missing values: 0
```

6.4 Checking Duplicate Student IDs

Each student should have a unique Student ID. Duplicate IDs can cause confusion when identifying students or analyzing their records.

We will use Pandas to check whether any Student IDs appear more than once. If the dataset is correct, there should be zero duplicate IDs.

```
# Count duplicate student IDs
duplicate_ids = df["Student_ID"].duplicated().sum()

print("Number of duplicate student IDs:", duplicate_ids)
```

```
Number of duplicate student IDs: 0
```

6.5 Validating Marks and Attendance

In this project, each subject is marked out of 100, so valid marks must be between 0 and 100. Attendance is also represented as a percentage between 0 and 100.

We will check these ranges to ensure that the dataset contains valid numerical values before performing academic performance analysis.

```
# Define the subject columns
subject_columns = [
    "Python",
    "Mathematics",
    "AI",
    "Statistics",
    "Communication"
]

# Check whether all subject marks are between 0 and 100
marks_are_valid = df[subject_columns].apply(
    lambda column: column.between(0, 100).all()
).all()

# Check whether attendance is between 0 and 100
attendance_is_valid = df["Attendance"].between(0, 100).all()

print("All subject marks are valid:", marks_are_valid)
print("All attendance values are valid:", attendance_is_valid)
```

```
All subject marks are valid: True
All attendance values are valid: True
```

```
print("DATA QUALITY SUMMARY")
```

```
print("--------------------")
print("Total student records:", len(df))
print("Total columns:", len(df.columns))
print("Missing values:", df.isnull().sum().sum())
print("Duplicate student IDs:", df["Student_ID"].duplicated().sum())
print("Valid subject marks:", marks_are_valid)
print("Valid attendance:", attendance_is_valid)
```

```
DATA QUALITY SUMMARY
--------------------
Total student records: 500
Total columns: 9
Missing values: 0
Duplicate student IDs: 0
Valid subject marks: True
Valid attendance: True
```

Part 4: Student Performance Analysis

In this section, we use **Pandas** to calculate each student's total and average marks across five subjects: Python, Mathematics, AI, Statistics, and Communication.

- **Total Marks:** The sum of marks obtained in all five subjects. The maximum possible total is 500.
- **Average Marks:** Total marks divided by the number of subjects. The maximum possible average is 100.

These calculations help us understand each student's overall academic performance.

```
# Calculate total marks for each student
df["Total_Marks"] = df[subject_columns].sum(axis=1)

# Calculate average marks for each student
df["Average_Marks"] = df[subject_columns].mean(axis=1).round(2)

# Display the results for the first five students
display(
    df[["Student_ID", "Name", "Total_Marks", "Average_Marks"]].head()
)
```

	Student_ID	Name	Total_Marks	Average_Marks
0	STU0001	Diya Mishra	299	59.8
1	STU0002	Kavya Pawar	423	84.6
2	STU0003	Yash Khan	201	40.2
3	STU0004	Amit Pawar	255	51.0
4	STU0005	Deepak Shinde	206	41.2

Part 5: Grading System and Pass/Fail Status

In this section, we assign grades to students based on their average marks. We also determine whether each student has passed or failed.

Grading rules:

Average Marks	Grade
90–100	A+
80–89.99	A
70–79.99	B
60–69.99	C
50–59.99	D
Below 50	F

Pass/Fail rule: A student passes if they score at least 40 marks in every subject. If they score below 40 in even one subject, they fail.

These rules help us classify students according to their academic performance.

```
def calculate_grade(average):
    if average >= 90:
        return "A+"
    elif average >= 80:
        return "A"
    elif average >= 70:
        return "B"
    elif average >= 60:
        return "C"
    elif average >= 50:
        return "D"
    else:
        return "F"


# Test the function with sample averages
print("Average 95:", calculate_grade(95))
print("Average 85:", calculate_grade(85))
print("Average 65:", calculate_grade(65))
print("Average 35:", calculate_grade(35))
```

```
Average 95: A+
Average 85: A
Average 65: C
Average 35: F
```

```
# Assign a grade to every student
df["Grade"] = df["Average_Marks"].apply(calculate_grade)

# Check whether each student passed all subjects
df["Result"] = np.where(
    df[subject_columns].ge(40).all(axis=1),
    "Pass",
    "Fail"
)

# Display student performance
display(
    df[
        [
            "Student_ID",
            "Name",
            "Average_Marks",
            "Grade",
            "Result"
        ]
    ].head(10)
```

```
)
```

	Student_ID	Name	Average_Marks	Grade	Result
0	STU0001	Diya Mishra	59.8	D	Pass
1	STU0002	Kavya Pawar	84.6	A	Pass
2	STU0003	Yash Khan	40.2	F	Fail
3	STU0004	Amit Pawar	51.0	D	Pass
4	STU0005	Deepak Shinde	41.2	F	Fail
5	STU0006	Meera Bhosale	87.0	A	Pass
6	STU0007	Yash Kadam	59.4	D	Pass
7	STU0008	Rahul Chavan	67.0	C	Pass
8	STU0009	Aditya Verma	34.2	F	Fail
9	STU0010	Neha Mane	70.2	B	Pass

Part 6: Class Performance Summary

In this section, we analyze the overall academic results of the students.

We will calculate:

- Total number of students who passed.
- Total number of students who failed.
- Overall pass and fail percentages.
- Number of students in each grade category.

This analysis helps us understand the overall performance of the class.

```
# Count students who passed and failed
pass_count = (df["Result"] == "Pass").sum()
fail_count = (df["Result"] == "Fail").sum()

# Calculate percentages
total_students = len(df)

pass_percentage = (pass_count / total_students) * 100
fail_percentage = (fail_count / total_students) * 100

# Display the summary
print("Total Students:", total_students)
print("Students Passed:", pass_count)
print("Students Failed:", fail_count)
print("Pass Percentage:", round(pass_percentage, 2), "%")
print("Fail Percentage:", round(fail_percentage, 2), "%")
```

```
Total Students: 500
Students Passed: 375
Students Failed: 125
Pass Percentage: 75.0 %
Fail Percentage: 25.0 %
```

```
# Count the number of students in each grade
grade_distribution = df["Grade"].value_counts().reindex(
    ["A+", "A", "B", "C", "D", "F"],
```

```
    fill_value=0
)

print("Grade Distribution:")
display(grade_distribution.to_frame(name="Number_of_Students"))
```

```
Grade Distribution:
```

Grade	Number_of_Students
A+	16
A	94
B	71
C	165
D	38
F	116

Part 7: Top Performers and Performance Ranking

In this section, we identify students based on their total and average marks.

We will:

- Find the student with the highest average marks.
- Find the student with the lowest average marks.
- Display the top 10 performing students.
- Rank students according to their average marks.

This analysis helps identify high achievers and students who may need additional academic support.

```
# Find the index of the highest and lowest average marks
highest_index = df["Average_Marks"].idxmax()
lowest_index = df["Average_Marks"].idxmin()

# Get the corresponding student records
highest_performer = df.loc[highest_index]
lowest_performer = df.loc[lowest_index]

print("Highest Performer:")
display(
    highest_performer[
        ["Student_ID", "Name", "Department", "Average_Marks", "Grade"]
    ].to_frame().T
)

print("Lowest Performer:")
display(
    lowest_performer[
        ["Student_ID", "Name", "Department", "Average_Marks", "Grade"]
    ].to_frame().T
)
```

Highest Performer:

	Student_ID	Name	Department	Average_Marks	Grade
265	STU0266	Arjun Kadam	Computer Engineering	94.4	A+

Lowest Performer:

	Student_ID	Name	Department	Average_Marks	Grade
248	STU0249	Riya Mehta	ENTC	27.8	F

Display the top 10 students

```
# Sort students from highest to lowest average marks
ranked_students = df.sort_values(
    by=["Average_Marks", "Student_ID"],
    ascending=[False, True]
).copy()

# Assign rankings
ranked_students["Rank"] = range(1, len(ranked_students) + 1)

# Display the top 10 performers
top_10_students = ranked_students[
    [
        "Rank",
        "Student_ID",
        "Name",
        "Department",
        "Total_Marks",
        "Average_Marks",
        "Grade",
        "Result"
    ]
].head(10)

display(top_10_students)
```

	Rank	Student_ID	Name	Department	Total_Marks	Average_Marks	Grade
265	1	STU0266	Arjun Kadam	Computer Engineering	472	94.4	A+
80	2	STU0081	Isha Kulkarni	Computer Engineering	468	93.6	A+
223	3	STU0224	Sakshi More	ENTC	464	92.8	A+
447	4	STU0448	Rohan Nair	Computer Engineering	464	92.8	A+
67	5	STU0068	Neha Mishra	Information Technology	460	92.0	A+
419	6	STU0420	Amit Sharma	ENTC	460	92.0	A+
			Pranav	Computer			

Part 8: Subject-wise Performance Analysis

In this section, we analyze the marks obtained in each subject by all students.

We will calculate:

- Average marks for each subject.
- Highest marks obtained in each subject.
- Lowest marks obtained in each subject.
- The subject with the highest and lowest class average.

This analysis helps us understand the subjects in which students perform well and the subjects that may require additional attention.

```
# Calculate average, highest, and lowest marks for each subject
subject_analysis = pd.DataFrame({
    "Average_Marks": df[subject_columns].mean().round(2),
    "Highest_Marks": df[subject_columns].max(),
    "Lowest_Marks": df[subject_columns].min()
})

print("Subject-wise Performance:")
display(subject_analysis)
```

```
Subject-wise Performance:
```

	Average_Marks	Highest_Marks	Lowest_Marks
Python	64.57	100	12
Mathematics	64.72	100	15
AI	64.73	100	19
Statistics	64.65	100	17
Communication	64.78	100	11

```
# Identify subjects with the highest and lowest average marks
highest_average_subject = subject_analysis["Average_Marks"].idxmax()
lowest_average_subject = subject_analysis["Average_Marks"].idxmin()

print(
    "Subject with the highest average:",
    highest_average_subject,
    "-",
    subject_analysis.loc[highest_average_subject, "Average_Marks"]
)

print(
    "Subject with the lowest average:",
    lowest_average_subject,
    "-",
    subject_analysis.loc[lowest_average_subject, "Average_Marks"]
)
```

```
Subject with the highest average: Communication - 64.78
Subject with the lowest average: Python - 64.57
```

Part 9: Data Visualization

Data visualization presents numerical information in graphical form, making patterns and comparisons easier to understand.

In this section, we will create:

- A bar chart comparing average marks across subjects.

- A pie chart showing the distribution of pass and fail results.
- A histogram showing the distribution of students' average marks.

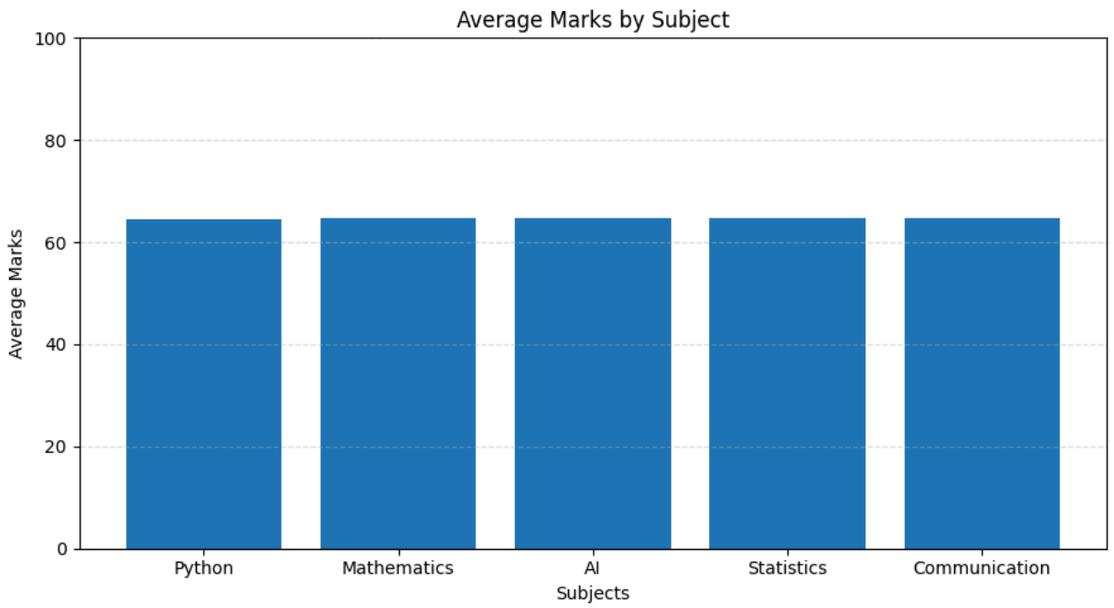
These charts help us interpret student performance more clearly.

```
import matplotlib.pyplot as plt
```

```
# Bar chart of average marks in each subject
plt.figure(figsize=(9, 5))

plt.bar(
    subject_analysis.index,
    subject_analysis["Average_Marks"]
)

plt.title("Average Marks by Subject")
plt.xlabel("Subjects")
plt.ylabel("Average Marks")
plt.ylim(0, 100)
plt.grid(axis="y", linestyle="--", alpha=0.4)
plt.tight_layout()
plt.show()
```



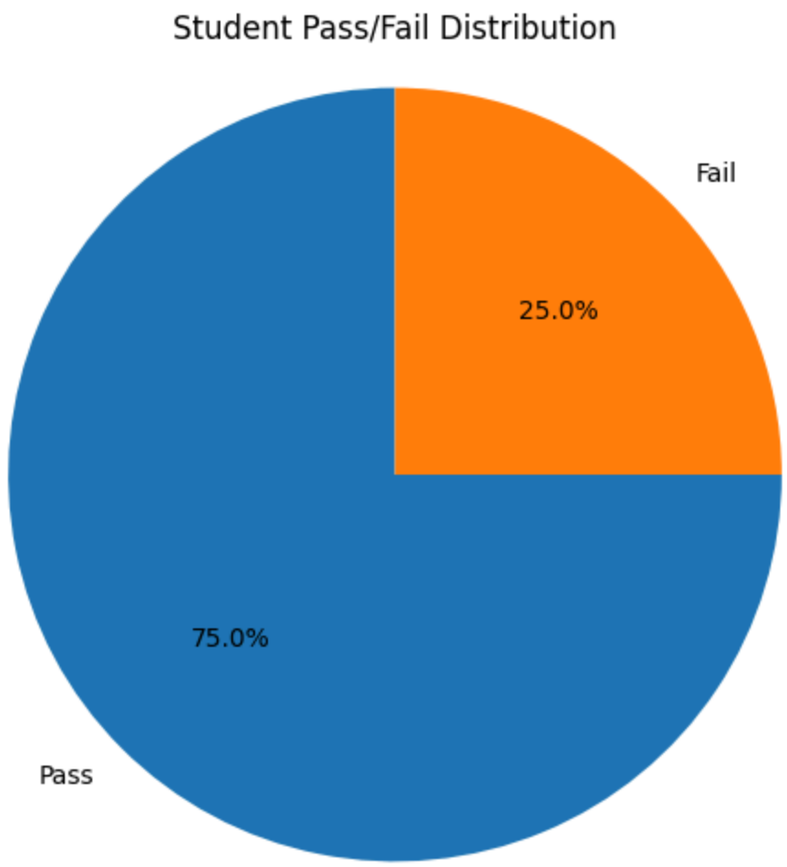

```
# Pie chart showing pass and fail distribution
result_counts = df["Result"].value_counts().reindex(
    ["Pass", "Fail"],
    fill_value=0
)

plt.figure(figsize=(6, 6))

plt.pie(
```

```
    result_counts,
    labels=result_counts.index,
    autopct="%1.1f%%",
    startangle=90
)

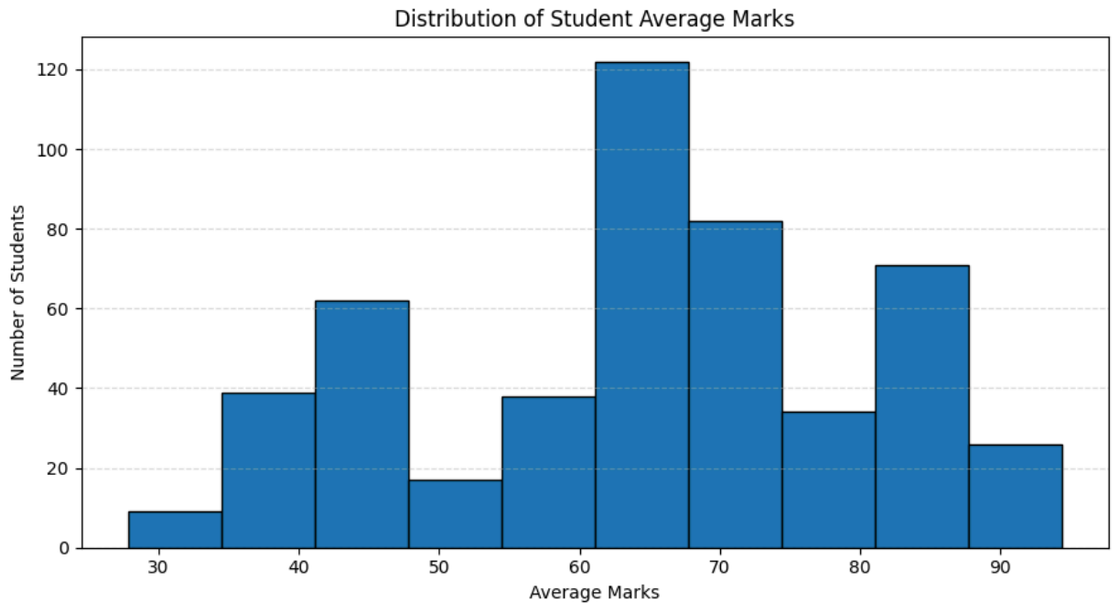
plt.title("Student Pass/Fail Distribution")
plt.axis("equal")
plt.show()
```




```
# Histogram of student average marks
plt.figure(figsize=(9, 5))

plt.hist(
    df["Average_Marks"],
    bins=10,
    edgecolor="black"
)

plt.title("Distribution of Student Average Marks")
plt.xlabel("Average Marks")
plt.ylabel("Number of Students")
plt.grid(axis="y", linestyle="--", alpha=0.4)
plt.tight_layout()
plt.show()
```



Part 10: Exporting Reports to CSV

After analyzing student performance, we save the results in CSV format.

The final student report will include student details, subject marks, total marks, average marks, grades, pass/fail status, and ranking.

CSV files can be opened in Microsoft Excel, Google Sheets, or other data-analysis tools. They also make it easy to share and reuse the results.

```
# Save the complete student performance report
final_report_path = project / "student_performance_report.csv"

# Add the student ranking to the main dataset
df["Rank"] = df["Average_Marks"].rank(
    method="first",
    ascending=False
).astype(int)

# Save the report
df.to_csv(final_report_path, index=False)

print("Student performance report saved successfully!")
print("File location:", final_report_path)
print("Total student records:", len(df))
```

```
Student performance report saved successfully!
File location: /content/Student_Performance_Analytics/student_performance_rep
Total student records: 500
```

```
# Save subject-wise analysis
subject_report_path = project / "subject_performance_report.csv"
subject_analysis.to_csv(subject_report_path, index_label="Subject")
```

```
# Save grade distribution
grade_report_path = project / "grade_distribution_report.csv"
grade_distribution.to_frame(
    name="Number_of_Students"
).to_csv(grade_report_path, index_label="Grade")

print("All reports saved successfully!")
print("\nFiles created:")

for file_path in [
    final_report_path,
    subject_report_path,
    grade_report_path
]:
    print("-", file_path.name)
```

```
All reports saved successfully!

Files created:
- student_performance_report.csv
- subject_performance_report.csv
- grade_distribution_report.csv
```

```
# Read the saved report back into Pandas
verified_report = pd.read_csv(final_report_path)

print("Report verification")
print("Rows:", verified_report.shape[0])
print("Columns:", verified_report.shape[1])
print("File exists:", final_report_path.exists())

display(verified_report.head())
```

```
Report verification
Rows: 500
Columns: 14
File exists: True
```

	Student_ID	Name	Department	Python	Mathematics	AI	Statistics	Commur
0	STU0001	Diya Mishra	ENTC	55	55	78	61	
1	STU0002	Kavya Pawar	Computer Engineering	89	67	87	92	
2	STU0003	Yash Khan	Computer Engineering	27	46	39	37	

Double-click (or enter) to edit

```
# Display all 500 student records
pd.set_option("display.max_rows", 500)
pd.set_option("display.max_columns", None)
pd.set_option("display.width", None)
```

```
display(df)
```

	Student_ID	Name	Department	Python	Mathematics	AI	Statistics
0	STU0001	Diya Mishra	ENTC	55	55	78	61
1	STU0002	Kavya Pawar	Computer Engineering	89	67	87	92
2	STU0003	Yash Khan	Computer Engineering	27	46	39	37
3	STU0004	Amit Pawar	Computer Engineering	62	56	41	46
4	STU0005	Deepak Shinde	Computer Engineering	43	51	41	34
5	STU0006	Meera Bhosale	ENTC	84	88	91	80
6	STU0007	Yash Kadam	Information Technology	52	82	60	48
7	STU0008	Rahul Chavan	Information Technology	67	63	62	73
8	STU0009	Aditya Verma	Information Technology	47	49	19	38
9	STU0010	Neha Mane	Computer Engineering	66	78	79	68
10	STU0011	Isha Gaikwad	Information Technology	36	42	53	36
11	STU0012	Vedant Mehta	Information Technology	55	77	74	67
12	STU0013	Karan Thakur	Information Technology	78	90	73	89
13	STU0014	Vedant Shinde	Computer Engineering	67	93	51	82
14	STU0015	Deepak Jadhav	ENTC	85	89	82	100
15	STU0016	Aditya Jadhav	ENTC	35	41	30	47
16	STU0017	Asha Mehta	Information Technology	64	54	63	48
17	STU0018	Priya Kulkarni	Information Technology	57	73	36	58
18	STU0019	Sneha Gaikwad	Information Technology	71	67	67	62
19	STU0020	Meera Pillai	Information Technology	49	39	35	42

```
# Load the complete report
all_students = pd.read_csv(
    "/content/Student_Performance_Analytics/student_performance_report.csv"
)

# Display all records
display(all_students)
```

	Student_ID	Name	Department	Python	Mathematics	AI	Statistics
24	STU0025	Yash More	Information Technology	88	73	89	84

```
# Display students numbered 1 to 50
display(df.iloc[0:50])
```

0	STU0001	Diya Mishra	ENTC	55	55	78	61
1	STU0002	Kavya Pawar	Computer Engineering	89	67	87	92
2	STU0003	Yash Khan	Computer Engineering	27	46	39	37
3	STU0004	Amit Pawar	Computer Engineering	62	56	41	46
4	STU0005	Deepak Shinde	Computer Engineering	43	51	41	34
5	STU0006	Meera Bhosale	ENTC	84	88	91	80
6	STU0007	Yash Kadam	Information Technology	52	82	60	48
7	STU0008	Rahul Chavan	Information Technology	67	63	62	73
8	STU0009	Aditya Verma	Information Technology	47	49	19	38
9	STU0010	Neha Mane	Computer Engineering	66	78	79	68
10	STU0011	Isha Gaikwad	Information Technology	36	42	53	36
11	STU0012	Vedant Mehta	Information Technology	55	77	74	67
12	STU0013	Karan Thakur	Information Technology	78	90	73	89
13	STU0014	Vedant Shinde	Computer Engineering	67	93	51	82
14	STU0015	Deepak Jadhav	ENTC	85	89	82	100
15	STU0016	Aditya Aniket Jadhav	ENTC	35	41	30	47
16	STU0017	Asha Mehta	Information Technology	64	54	63	48
17	STU0018	Priya Vikram Kulkarni	Information Technology	57	73	36	58
18	STU0019	Sneha Gaikwad	Information Technology	71	67	67	62
19	STU0020	Meera Bhosale	Information Technology	49	39	35	42
20	STU0021	Riya Joshi	ENTC	42	56	47	51
21	STU0022	Kavya Kulkarni	Computer Engineering	56	63	58	61
22	STU0023	Diya Gaikwad	Information Technology	69	75	98	93
23	STU0024	Yash Riya Gaikwad	ENTC	53	63	67	67
24	STU0025	Yash More	Information Technology	88	73	89	84
25	STU0026	Neha Deepak Verma	ENTC	53	63	67	67
26	STU0027	Rohan Kulkarni	Computer Engineering	56	63	58	61
27	STU0028	Kadam Khan	Information Technology	93	84	100	79
28	STU0029	Chavan	Engineering	20	36	42	50
29	STU0030	Deepak	Information	58	68	68	48
30	STU0031	Kulkarni	Engineering	74	67	66	82
31	STU0032	Nikhil	Information	84	88	91	80
32	STU0033	Kadam	Engineering	100	100	79	80
33	STU0034	Aditya	ENTC	68	78	68	68
34	STU0035	Shinde	Technology	70	71	51	75
35	STU0036	Nikhil	Information	84	88	91	80
36	STU0037	Verma	Engineering	83	77	63	74
37	STU0038	Deepak	ENTC	58	68	68	54
38	STU0039	Chavan	ENTC	47	41	35	35
39	STU0040	Meera Iyer	ENTC	91	69	76	61
40	STU0041	Priya Bhosale	Engineering	77	67	61	46
41	STU0042	Aniket Joshi	ENTC	42	56	47	51
42	STU0043	Vikram Bhosale	Engineering	53	67	70	57
43	STU0044	Vikram Kulkarni	Computer Engineering	53	67	70	57
44	STU0045	Riya Gaikwad	ENTC	53	63	67	67
45	STU0046	Priya Bhosale	Information Technology	77	67	61	46
46	STU0047	Kavya Kulkarni	Computer Engineering	56	63	58	61
47	STU0048	Kadam Khan	Technology	93	84	100	79
48	STU0049	Diya Gaikwad	Information Technology	69	75	98	93
49	STU0050	Yash Riya Gaikwad Bhosale	ENTC	49	49	59	58

	Student_ID	Name	Department	Python	Mathematics	AI	Statistics
0	STU0001	Diya Mishra	ENTC	55	55	78	61
1	STU0002	Aniket Pawar	Computer Engineering	89	67	87	92
2	STU0003	Yash Khan	Computer Engineering	27	46	39	37
3	STU0004	Pranav Meera	Computer Engineering	62	56	41	46
4	STU0005	Deepak Shinde	Computer Engineering	43	51	41	34
5	STU0006	Pooja Bhosale	Computer Engineering	84	88	91	80
6	STU0007	Yash Kadam	Information Technology	52	82	60	48
7	STU0008	Rahul Chavan	Information Technology	67	63	62	73
8	STU0009	Aditya Verma	Information Technology	47	49	19	38
9	STU0010	Neha Mane	Computer Engineering	66	78	79	68
10	STU0011	Isha Gaikwad	Information Technology	36	42	53	36
11	STU0012	Aditya Mehta	Information Technology	55	77	74	67
12	STU0013	Karan Thakur	Information Technology	78	90	73	89
13	STU0014	Vedant Sameer	Computer Engineering	67	93	51	82
14	STU0015	Deepak Jadhav	ENTC	85	89	82	100
15	STU0016	Aditya Bhavana	ENTC	35	41	30	47
16	STU0017	Asha Mehta	Information Technology	64	54	63	48
17	STU0018	Priya Kavya	Information Technology	57	73	36	58
18	STU0019	Sneha Gaikwad	Information Technology	71	67	67	62
19	STU0020	Meera Yash	Information Technology	49	39	35	42
20	STU0021	Kavya Anjali	Computer Engineering	81	42	44	35
21	STU0022	Varun Mehta	Computer Engineering	79	42	44	35
22	STU0023	Diya More	ENTC	66	65	66	66
23	STU0024	Varun Deshmukh	Computer Engineering	42	44	35	35
24	STU0025	Yash More	Information Technology	88	73	89	84
25	STU0026	Priya Mehta	ENTC	82	81	100	83
26	STU0027	Yash Khan	Computer Engineering	79	42	44	35
27	STU0028	Diya More	ENTC	67	66	65	66
28	STU0029	Kavya Pawar	Computer Engineering	89	67	87	92
29	STU0030	Nikhil Nair	Information Technology	77	77	68	77
30	STU0031	Amit Pawar	Computer Engineering	62	56	41	46
31	STU0032	Meera Iyer	Computer Engineering	53	62	63	49
32	STU0033	Pooja Bhosale	ENTC	84	88	91	80
33	STU0034	Aditya Bhosale	ENTC	83	73	83	66
34	STU0035	Diva Chavan	Information Technology	67	63	62	73
35	STU0036	Neha Mane	Information Technology	84	69	81	88
36	STU0037	Akash Mane	Computer Engineering	66	78	79	68
37	STU0038	Deepak Deshmukh	ENTC	52	56	82	54
38	STU0039	Vedant Iyer	Information Technology	55	77	74	67
39	STU0040	Aditya Krishna	Information Technology	78	90	73	89
40	STU0041	Sameer Khan	ENTC	37	51	46	41
41	STU0042	Bhavana Iyer	Information Technology	50	36	48	48
42	STU0043	Jadhav Iyer	Information Technology	50	36	48	48
43	STU0044	Kavya Kulkarni	Information Technology	73	83	80	55
44	STU0045	Deshmukh Iyer	Technology	73	83	80	55
45	STU0046	Priya Patil	Computer Engineering	85	100	75	87
46	STU0047	Patil Mehta	Technology	85	100	75	87
47	STU0048	Jadhav Gaikwad	Computer Engineering	59	66	59	84
48	STU0049	Gaikwad Jadhav	ENTC	59	66	59	84
49	STU0050	Jadhav Mehta	Computer Engineering	59	66	59	84
50	STU0051	Gaikwad Jadhav	ENTC	59	66	59	84

```
# Display all files saved in the project folder
print("Project folder:", project)
print("\nFiles available:")

for file_path in sorted(project.iterdir()):
    if file_path.is_file():
        print(f"{file_path.name} — {file_path.stat().st_size:,} bytes")
```

```
Project folder: /content/Student_Performance_Analytics

Files available:
```

```
README.md — 0 bytes
functions.py — 3 bytes
grade_distribution_report.csv — 58 bytes
main.py — 0 bytes
student_performance_report.csv — 37,685 bytes
students.csv — 27,733 bytes
subject_performance_report.csv — 160 bytes
```

Reusable Student Performance Function

A reusable function allows us to perform the same calculation for different students without repeating the code.

In this section, we use NumPy to calculate total and average marks for a student's five subjects. The function returns both values for further analysis.

```
def calculate_student_performance(marks):
    """Calculate total and average marks for one student."""
    marks = np.asarray(marks, dtype=float)

    total = np.sum(marks)
    average = np.mean(marks)

    return int(total), round(float(average), 2)


# Test the function using the first student's subject marks
sample_marks = df.loc[0, subject_columns].to_numpy()

total, average = calculate_student_performance(sample_marks)

print("Total Marks:", total)
print("Average Marks:", average)
```

```
Total Marks: 299
Average Marks: 59.8
```

```
import pandas as pd
import numpy as np
from pathlib import Path

project = Path("/content/Student_Performance_Analytics")
df = pd.read_csv(project / "students.csv")

subject_columns = [
    "Python",
    "Mathematics",
    "AI",
    "Statistics",
    "Communication"
]

print("Student records loaded:", len(df))
```

```
Student records loaded: 500
```

Overall Class Performance Summary

In this section, we calculate the overall class average, the highest student average, and the lowest student average.

These statistics help summarize the academic performance of all students in the dataset.-

78	STU0079	Pooja Singh	ENTC	51	82	64	69
104	STU0105	Mane	ENTC	83	54	65	78
39	STU0040	Asha Sharma	Information Technology	89	35	88	48
80	STU0081	Pranav Isha Kulkarni	Computer Computer Engineering	100	83	100	85
92	STU0093	Thakur	Engineering	79	64	43	73
93	STU0094	Isha Pillai Varun	ENTC	77	53	88	55
94	STU0095	Bhavana More	Information Technology	74	100	81	93
120	STU0121	Vikram More	Technology	68	52	71	61

```
# Load the compiled report containing the calculated columns (such as Averag
df_report = pd.read_csv(project / "student_performance_report.csv")

# Calculate overall class performance
class_average = df_report["Average_Marks"].mean()
highest_average = df_report["Average_Marks"].max()
lowest_average = df_report["Average_Marks"].min()

# Identify the students with the highest and lowest averages
highest_student = df_report.loc[df_report["Average_Marks"].idxmax()]
lowest_student = df_report.loc[df_report["Average_Marks"].idxmin()]

# Display the summary
print("OVERALL CLASS PERFORMANCE")
print("-------------------------")
print("Total Students:", len(df_report))
print("Class Average:", round(class_average, 2))
print(
    "Highest Performer:",
    highest_student["Name"],
    "-",
    highest_student["Average_Marks"]
)
print(
    "Lowest Performer:",
    lowest_student["Name"],
    "-",
    lowest_student["Average_Marks"]
)
```

```
OVERALL CLASS PERFORMANCE
-------------------------
Total Students: 500
Class Average: 64.69
Highest Performer: Arjun Kadam - 94.4
Lowest Performer: Riya Mehta - 27.8
```

```
# Count pass and fail students using the correct report DataFrame
pass_count = (df_report["Result"] == "Pass").sum()
fail_count = (df_report["Result"] == "Fail").sum()

# Calculate percentages
total_students = len(df_report)
pass_percentage = (pass_count / total_students) * 100
fail_percentage = (fail_count / total_students) * 100

print("PASS/FAIL SUMMARY")
print("-----------------")
print("Total Students:", total_students)
print("Passed:", pass_count)
print("Failed:", fail_count)
print("Pass Percentage:", round(pass_percentage, 2), "%")
print("Fail Percentage:", round(fail_percentage, 2), "%")

# Display grade distribution
print("\nGRADE DISTRIBUTION")
grade_counts = df_report["Grade"].value_counts().reindex(
    ["A+", "A", "B", "C", "D", "F"],
    fill_value=0
```

```
)

display(grade_counts.to_frame(name="Number_of_Students"))
```

```
PASS/FAIL SUMMARY
----------------------------------------
Total Students: 500
Passed: 375
Failed: 125
Pass Percentage: 75.0%
Fail Percentage: 25.0%

GRADE DISTRIBUTION
```

Grade	Number_of_Students
A+	16
A	94
B	71
C	165
D	38
F	116

```
from pathlib import Path
import pandas as pd

# Create the project folder if needed
project = Path("/content/Student_Performance_Analytics")
project.mkdir(parents=True, exist_ok=True)

# Save the complete student report
final_report_path = project / "student_performance_report.csv"
df.to_csv(final_report_path, index=False)

# Save subject-wise analysis if it exists
if "subject_analysis" in globals():
    subject_analysis.to_csv(
        project / "subject_performance_report.csv",
        index_label="Subject"
    )

# Save grade distribution
if "Grade" in df.columns:
    grade_counts = df["Grade"].value_counts().reindex(
        ["A+", "A", "B", "C", "D", "F"],
        fill_value=0
    )
    grade_counts.to_frame(
        name="Number_of_Students"
    ).to_csv(project / "grade_distribution_report.csv",
             index_label="Grade")

# Verify the saved student report
verified_report = pd.read_csv(final_report_path)

print("Report saved successfully:", final_report_path.exists())
print("Number of student records:", len(verified_report))
print("Number of columns:", len(verified_report.columns))
print("\nAvailable columns:")
print(verified_report.columns.tolist())
```

```
display(verified_report.head())
```

```
Report saved successfully: True
Number of student records: 500
Number of columns: 9
Available columns: ['Student_ID', 'Name', 'Department', 'Python', 'Mathematics', 'AI', 'Statisti
```

	Student_ID	Name	Department	Python	Mathematics	AI	Statistics	Commur
0	STU0001	Diya Mishra	ENTC	55	55	78	61	
1	STU0002	Kavya Kadam	Computer Engineering	89	67	87	92	
2	STU0003	Yash Khan	Computer Engineering	27	46	39	37	

```
# Check the main project requirements
required_columns = [
    "Student_ID",
    "Name",
    "Department",
    "Python",
    "Mathematics",
    "AI",
    "Statistics",
    "Communication",
    "Attendance",
    "Total_Marks",
    "Average_Marks",
    "Grade",
    "Result"
]

print("FINAL PROJECT VALIDATION")
print("-------------------------")
print("Total student records:", len(df))

print("\nRequired column checks:")
for column in required_columns:
    status = "PASS" if column in df.columns else "MISSING"
    print(f"{column}: {status}")

print("\nDuplicate student IDs:", df["Student_ID"].duplicated().sum())
print("Missing values in dataset:", df.isnull().sum().sum())

# Verify that the dataset has the expected number of records
print("\nContains 500 students:", len(df) == 500)
```

```
FINAL PROJECT VALIDATION
-------------------------
Total student records: 500

Required column checks:
Student_ID: PASS
Name: PASS
Department: PASS
Python: PASS
Mathematics: PASS
AI: PASS
Statistics: PASS
Communication: PASS
```

```
Attendance: PASS
Total_Marks: MISSING
Average_Marks: MISSING
Grade: MISSING
Result: MISSING
Duplicate student IDs: 0
Missing values in dataset: 0
Contains 500 students: True
```

158	STU0159	Bhavana Jadhav	Information Technology	26	30	53	21
160	STU0161	Diya Pillai	Computer Engineering	80	75	98	69

```
# Define the subject columns
subject_columns = [
    "Python",
    "Mathematics",
    "AI",
    "Statistics",
    "Communication"
]

# 1. Calculate total marks for each student
df["Total_Marks"] = df[subject_columns].sum(axis=1)

# 2. Calculate average marks for each student
df["Average_Marks"] = df[subject_columns].mean(axis=1).round(2)

# 3. Assign grades
def calculate_grade(average):
    if average >= 90:
        return "A+"
    elif average >= 80:
        return "A"
    elif average >= 70:
        return "B"
    elif average >= 60:
        return "C"
    elif average >= 50:
        return "D"
    else:
        return "F"

df["Grade"] = df["Average_Marks"].apply(calculate_grade)

# 4. Assign pass/fail status
# Pass requires at least 40 marks in every subject
df["Result"] = np.where(
    df[subject_columns].ge(40).all(axis=1),
    "Pass",
    "Fail"
)

print("Calculated columns added successfully!")
display(
    df[
        [
            "Student_ID",
            "Total_Marks",
            "Average_Marks",
            "Grade",
            "Result"
        ]
    ].head()
)
```

```
Calculated columns added successfully!
```

	Student_ID	Total_Marks	Average_Marks	Grade	Result			
184	STU0185	Vedant Gupta	Computer Engineering	78		70	84	85
0	STU0001	299	59.8	D	Pass			
185	STU0186	Mishra	Technology	70		70	69	68
1	STU0002	423	84.6	A	Pass			
2	STU0003	201	40.2	F	Fail	51	59	71
3	STU0004	255	51.0	D	Pass			
318	STU0319	Vikram	Computer	51		76	71	59
4	STU0005	206	41.2	F	Fail	54	43	54
188	STU0189	Aditya	ENTC	64		100	87	65

```
# Save the updated student performance report
final_report_path = project / "student_performance_report.csv"
df.to_csv(final_report_path, index=False)

print("Updated report saved:", final_report_path.exists())
print("Total student records:", len(df))
print("Total columns:", len(df.columns))
```

192	STU0193	Nair	Engineering	75		85	84	86

```
Updated report saved: True
Total student records: 500
Total columns: 13
```

248	STU0249	Gaikar	Computer	60		44	80	66
498	STU0499	More	ENTC	71		78	67	92

```
readme_content = """# Student Performance Analytics System

## Project Overview
A Python-based project that analyzes student academic performance
using Pandas and NumPy.

## Features
- Student dataset with marks and attendance
- Total and average marks calculation
- Grade assignment using defined rules
- Pass/fail classification
- Overall class performance analysis
- Subject-wise performance analysis
- Top 10 student ranking
- Charts and CSV reports

## Technologies Used
- Python
- NumPy
- Pandas
- Matplotlib

## Dataset
The dataset contains 500 student records, including student ID,
name, department, five subject marks, and attendance.

## Grading Rules
- 90 and above: A+
- 80 to below 90: A
- 70 to below 80: B
- 60 to below 70: C
- 50 to below 60: D
- Below 50: F

## Pass/Fail Rule
A student passes if they score at least 40 marks in every subject.

## How to Run
```

```
Open the project notebook in Google Colab and run the cells
in order, starting from the imports and dataset setup.
"""

readme_path = project / "README.md"
readme_path.write_text(readme_content, encoding="utf-8")

print("README created:", readme_path.exists())
```

```
README created: True
```

```
print("PROJECT FILES")
print("-------------")

for file_path in sorted(project.iterdir()):
    if file_path.is_file():
        print(file_path.name)
```

```
PROJECT FILES
-------------
README.md
functions.py
grade_distribution_report.csv
main.py
student_performance_report.csv
students.csv
subject_performance_report.csv
```

```
main_path = project / "main.py"

print("main.py exists:", main_path.exists())
print("File size:", main_path.stat().st_size, "bytes")

print("\nFirst 40 lines of main.py:")
print("\n".join(main_path.read_text(encoding="utf-8").splitlines()[:40]))
```

```
main.py exists: True
File size: 0 bytes

First 40 lines of main.py:
```

```
main_code = r'''
from pathlib import Path
import numpy as np
import pandas as pd

PROJECT_DIR = Path(__file__).resolve().parent
DATA_FILE = PROJECT_DIR / "students.csv"
SUBJECTS = [
    "Python", "Mathematics", "AI",
    "Statistics", "Communication"
]


def calculate_grade(average):
    """Return a grade based on the student's average marks."""
    if average >= 90:
        return "A+"
    elif average >= 80:
```

```
        return "A"
    elif average >= 70:
        return "B"
    elif average >= 60:
        return "C"
    elif average >= 50:
        return "D"
    return "F"


def calculate_student_performance(marks):
    """Calculate total and average marks."""
    marks = np.asarray(marks, dtype=float)
    return int(np.sum(marks)), round(float(np.mean(marks)), 2)


def analyze_students(df):
    """Calculate student results and create analysis reports."""
    df = df.copy()

    df["Total_Marks"] = df[SUBJECTS].sum(axis=1)
    df["Average_Marks"] = df[SUBJECTS].mean(axis=1).round(2)
    df["Grade"] = df["Average_Marks"].apply(calculate_grade)

    df["Result"] = np.where(
        df[SUBJECTS].ge(40).all(axis=1),
        "Pass",
        "Fail"
    )

    df["Rank"] = df["Average_Marks"].rank(
        method="first", ascending=False
    ).astype(int)

    return df


def main():
    if not DATA_FILE.exists():
        print("Error: students.csv was not found.")
        return

    df = pd.read_csv(DATA_FILE)

    missing_columns = [
        column for column in
        ["Student_ID", "Name", "Department", "Attendance"] + SUBJECTS
        if column not in df.columns
    ]

    if missing_columns:
        print("Missing required columns:", missing_columns)
        return

    df = analyze_students(df)

    output_file = PROJECT_DIR / "student_performance_report.csv"
    df.to_csv(output_file, index=False)

    subject_report = pd.DataFrame({
        "Average_Marks": df[SUBJECTS].mean().round(2),
        "Highest_Marks": df[SUBJECTS].max(),
        "Lowest_Marks": df[SUBJECTS].min()
    })
    subject_report.to_csv(
        PROJECT_DIR / "subject_performance_report.csv",
```

```
        index_label="Subject"
    )

    grade_counts = df["Grade"].value_counts().reindex(
        ["A+", "A", "B", "C", "D", "F"],
        fill_value=0
    )
    grade_counts.to_frame(
        name="Number_of_Students"
    ).to_csv(
        PROJECT_DIR / "grade_distribution_report.csv",
        index_label="Grade"
    )

    print("STUDENT PERFORMANCE ANALYTICS")
    print("-----------------------------")
    print("Total students:", len(df))
    print("Class average:", round(df["Average_Marks"].mean(), 2))
    print("Passed:", int((df["Result"] == "Pass").sum()))
    print("Failed:", int((df["Result"] == "Fail").sum()))

    print("\\nTop 10 students:")
    print(
        df.sort_values("Rank")[
            ["Rank", "Student_ID", "Name", "Average_Marks", "Grade", "Result
        ].head(10).to_string(index=False)
    )

    print("\\nSubject-wise analysis:")
    print(subject_report)

    print("\\nReports saved successfully.")


if __name__ == "__main__":
    main()
'''

main_path.write_text(main_code.strip() + "\n", encoding="utf-8")
print("main.py created successfully.")
print("File size:", main_path.stat().st_size, "bytes")
```

```
main.py created successfully.
File size: 3068 bytes
```

```
print("main.py size:", main_path.stat().st_size, "bytes")
print("\nFirst 10 lines:")
print("\n".join(main_path.read_text(encoding="utf-8").splitlines()[:10]))
```

```
main.py size: 3068 bytes

First 10 lines:
from pathlib import Path
import numpy as np
import pandas as pd

PROJECT_DIR = Path(__file__).resolve().parent
DATA_FILE = PROJECT_DIR / "students.csv"
SUBJECTS = [
    "Python", "Mathematics", "AI",
    "Statistics", "Communication"
]
```

```
import subprocess
import sys

result = subprocess.run(
    [sys.executable, str(main_path)],
    capture_output=True,
    text=True
)

print("Program exit code:", result.returncode)
print("\nPROGRAM OUTPUT:")
print(result.stdout)

if result.stderr:
    print("\nERROR MESSAGE:")
    print(result.stderr)
```

```
Program exit code: 0

PROGRAM OUTPUT:
STUDENT PERFORMANCE ANALYTICS
----------------------------------------
Total students: 500
Class average: 64.69
Passed: 375
Failed: 125

Top 10 students:
```

295	STU0294	Tanvi Gupta	Computer Engineering	62	57	71	65
296	STU0297	Pranav Jadhav	Computer Engineering	97	93	49	93
297	STU0298	Ashvi More	Computer Engineering	79	78	61	64
298	STU0299	Rahul Deshmukh	ENTC	51	77	60	62
299	STU0300	Yash Pillai	ENTC	38	24	49	94
300	STU0301	Sameer Patil	Information Technology	88	92	94	76